Project Abstract

An AI-Powered Mobile Web Application for Automated Dietary Tracking and Nutritional Analysis

Niraj Vaidya A70405223113
Diwakar Rai A70405223026
P Jaswant Rao A70405223030

ABSTRACT

Dietary tracking is often tedious and error-prone, requiring users to manually search databases, estimate portion sizes, and log macronutrients. This project introduces a **smart calorie and nutrition tracking application** built with modern web technologies—integrating a client-side AI vision pipeline, barcode recognition, and cloud-synchronised storage—to explore whether nutritional logging can be automated and streamlined for a superior user experience.

In **Stage 1**, the system leverages **on-device AI image classification** via **Transformers.js**—running pre-trained Hugging Face vision models directly in the browser—to predict the specific food item from a user-captured photo *before the user even types a word*, while also offering **barcode scanning** (via `html5-qrcode`) and **text search** as complementary low-friction logging paths.

In **Stage 2**, the system enriches the prediction with **real-time nutritional telemetry** from the **Open Food Facts** public database, fetching calories, protein, carbohydrates, and fats per 100 g/ml, then scaling them precisely to the user's chosen serving amount and unit before persisting the entry.

In **Stage 3**, the application layers **personalisation and gamification**: it auto-computes daily calorie goals from the user's body profile using the **Mifflin–St Jeor BMR equation** combined with an activity multiplier (TDEE) and a weight-goal offset; tracks **water intake**, **weight history**, and **consecutive-day streaks**; and presents progress through interactive **Recharts** visualisations on a dedicated dashboard.

This hybrid approach demonstrates the power of combining client-side AI inference, open public APIs, and a real-time cloud backend behind a single responsive frontend, showcasing how **integrated AI models and serverless cloud architecture** can be applied to everyday, high-friction domains like personal health—where accuracy and ease-of-use are paramount.

Flow Diagram Description

The prediction and tracking pipeline consists of **three major stages** connected sequentially:

1. **Stage 1: Multi-Modal Food Capture**
    - **Input Data:** Either (a) a user-captured image via the camera modal, (b) a product barcode scanned through the live camera feed, or (c) a free-text food query.
    - **Processing:**
        - **Image path** → preprocessing (compression, resizing) → on-device **Transformers.js** vision classification (Hugging Face food-recognition model) → top food prediction.
        - **Barcode path** → `html5-qrcode` decodes EAN/UPC → product key.
        - **Search path** → query string is forwarded directly to the nutrition layer.
    - **Output:** A canonical food identifier (predicted name, barcode, or search term).
2. **Stage 2: Nutritional Resolution & Serving Scaling**
    - **Input Data:** The **Open Food Facts** REST API returns the product's name, brand, and per-100 g/ml macronutrient profile (calories, protein, carbs, fat).
    - **Processing:** The user selects a meal slot (Breakfast/Lunch/Dinner/Snacks) and adjusts the serving amount and unit (g, ml, oz, serving). All macros are scaled client-side *before* writing to Firestore so that stored values are already serving-accurate.
    - **Output:** A finalised food entry persisted under `users/{uid}/logs/{YYYY-MM-DD}/entries/{entryId}`.
3. **Stage 3: Aggregation, Gamification & Presentation**
    - **Goal Engine:** The Settings page applies the **Mifflin–St Jeor** BMR formula, multiplies by the user's activity level for TDEE, and offsets by goal (`-500` lose / `0` maintain / `+500` gain) to auto-set `calorieGoal`.
    - **Streaks:** On every successful log, `lastLogDate` is compared with today; `currentStreak` increments on consecutive days, resets if a day is skipped, and surfaces as a 🔥 badge.
    - **Visualisation:** Data is displayed as an animated **Calorie Ring**, **Macro Bars**, a **Water Tracker**, collapsible **Meal Sections**, a **Recharts**-powered Dashboard (calories, macros, water, weight trends), and a **History** view of past days — all backed by Firebase Cloud Firestore for real-time multi-device sync.

Libraries and API's to be used

Frontend & State Management

- `Next.js 14 (App Router) / React` → component-based UI, file-system routing, and efficient client/server state handling.
- `CSS Modules + Global CSS Variables` → scoped styling with a unified design-token system (glassmorphism, dark-mode palette, backdrop filters, spring animations).
- `React Context (AuthContext)` → app-wide auth state via Firebase `onAuthStateChanged`.

Backend, Auth & Database

- `Firebase Authentication` → secure Google Sign-In as the sole identity provider.
- `Firebase Cloud Firestore` → real-time NoSQL persistence for user profile, daily logs, food entries, water intake, weight history, custom foods, and streaks.
- `Firebase App Hosting / Hosting` → production deployment target.

On-Device AI & Scanning

- `Transformers.js` (Hugging Face) → client-side food image classification, eliminating server-side inference cost.
- `html5-qrcode` → in-browser barcode (EAN/UPC) detection from the live camera stream.

Data Sources / APIs

- `Open Food Facts REST API` → free, open product database for barcode lookup and per-100 g/ml macronutrient data.

Data Visualisation

- `Recharts` → responsive line and bar charts for the Dashboard (calories, macros, water, weight).